

Lab 2 — Config-Driven Data Pipelines

Track A (tabular fraud-detection) · Week 2 · DS5619 Machine Learning Systems Operations

Setup

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number>
```

The last command overwrites `data/v1/transactions.csv` and `data/v1/transactions.json` with a dataset generated deterministically from your student ID — same shape as everyone else's (500 rows, same columns), different actual values. Two students never get the same data, and re-running with your own ID always reproduces the exact same files.

Note: `data/v1/transactions.parquet` and `data/v2/` may also be present but **not used by this lab** — ignore them (they support a later lab in the course).

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed under `data/`.

Files

- `src/pipeline_hardcoded.py` — the working-but-hardcoded starting point (given, don't edit).
- `src/pipeline.py` — implement the three `# TODO` functions here.
- `config/pipeline.example.yaml` — copy this to `config/pipeline.yaml` and fill in real values.
- `data/v1/transactions.csv`, `data/v1/transactions.json` — your personalized dataset, generated above (don't hand-edit).

Background

`src/pipeline_hardcoded.py` is a small, working script. It reads a transactions file, flags transactions above an amount threshold as "high value," and writes a summary report. It works — and it is exactly the kind of script the lecture's "Configuration Debt" slide warned about: the file path, the input format, and the dollar threshold are all literal values baked into the function bodies. Changing any of them means editing and redeploying code.

Run it once before you touch anything, so you know what correct output looks like:

```
python src/pipeline_hardcoded.py
cat data/v1/report_hardcoded.json
```

Your task

Fill in `src/pipeline.py`. It is scaffolded with the CLI argument already wired up — your job is the body of three functions, each marked `# TODO`:

1. `load_config(path)` — load and validate a YAML config file. It must contain `input_path`, `input_format` (`csv` or `json`), `high_value_threshold`, and `output_path`. Raise a clear `ValueError` naming the missing key if any are absent — don't let it fail with a bare `KeyError` three functions later.
2. `load_transactions(path, fmt)` — load transactions from either CSV or JSON into a list of dicts, based on `fmt`, not based on sniffing the file extension. This is the abstraction that makes format a config choice instead of a code branch someone has to remember to update.
3. `run_pipeline(config)` — wire the above together: load the config, load the data, compute the same summary as the hardcoded version (count, total amount, fraud rate, count of high-value transactions using the configured threshold), and write it to `config["output_path"]`.

Copy `config/pipeline.example.yaml` to `config/pipeline.yaml` and fill in real values pointing at `data/v1/transactions.csv`. Then run:

```
python src/pipeline.py --config config/pipeline.yaml
```

Prove the refactor actually removed the hardcoding: without touching `src/pipeline.py` again, create a second config file that points at `data/v1/transactions.json` (the same data, different format) and a different threshold, and run the pipeline against it. Both runs should succeed and produce sensible, different reports purely from the config change.

Self-check

```
pytest -q
```

This is a self-check, not the grader — passing it means your expectation functions and pipeline wiring work on the known cases, not that you're done.

Deliverables (what you commit)

- `src/pipeline.py` — completed, no `# TODO` markers left, no hardcoded paths or thresholds anywhere in the file.
- `config/pipeline.yaml` and a second config file (e.g. `config/pipeline_json.yaml`) demonstrating the format-swap.
- The two output reports your configs produced.
- `data/v1/transactions.csv` and `data/v1/transactions.json` as produced by YOUR `generate_for_student.py` run (see above) — commit them, they're small.
- A short `NOTES.md` (5-10 lines): the `--student-id` value you used (required — see above), plus *what specifically was hardcoded in the original script, and what would have had to happen to change the threshold or switch formats before your refactor?*

Grading checklist

- ☐ `data/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `pipeline.py` runs successfully against a CSV config.
- ☐ `pipeline.py` runs successfully against a JSON config, unmodified.
- ☐ No file paths, format strings, or threshold numbers are hardcoded in `pipeline.py`.
- ☐ `load_config` fails with a clear error message on a config missing a required key.
- ☐ `NOTES.md` correctly identifies the hardcoded values in the original script.
- ☐ Meaningful commit history (not one dump commit) and a working README.

Submission

Tag your final commit `week02-submit` and push it before the deadline:

```
git add -A
git commit -m "Week 2: config-driven pipeline"
git tag week02-submit
git push origin main --tags
```